

ADVANCED AUTOMATION ENGINEERING PROGRAM

Login Workflow

Locators · DOM · input handling · reliable interactions

SPRINT 1

DAY 2 of 42

Jai Singh

Week 1 — UI Automation (Playwright) · Full-day session (3h teach + 5h hands-on)

Planner ref: Week 1 · Day 2 — Login Workflow

01 How Today Runs

SESSION MAP

Day 1 launched and validated a page. Today we interact with it — find elements reliably, fill forms, automate a login. Most of the day is hands-on in the IDE.

09:30



Concepts

Locators, priority, DOM, lazy re-resolution

11:00



Locators & forms, live

Input handling, login automation — in the IDE

12:00



Guided exercises

Short 'you try' tasks with checkpoints

13:30



Lab (staged)

Build the login workflow in gated milestones

16:30



Debrief & recap

Common failures, evaluation, Day-3 preview

Today is about interacting with the page reliably and automating the first real user workflow — login. Synchronization (waits) is tomorrow.



Locators

Find elements like a user



Locator priority

role → label → text → testid → css → xpath



DOM basics

What you are selecting



Input handling

fill, check, select



Form automation

Submit & handle results



Login workflow

Today's lab

Aligned to Planner Wk1/Day2 “Login Workflow”. **ToC today:** locators (css/xpath/text/role), DOM, input handling, form automation, assertions. **Deferred:** frames/popups/multi-tab→D4, waits→D3.

By the end of today you will be able to:

- ✓ Explain what a locator is and how Playwright resolves it against the DOM.
- ✓ Choose locators in the recommended priority: role → label → text → test-id → CSS → XPath.
- ✓ Fill text inputs, check boxes, and select dropdown options reliably.
- ✓ Automate a complete login: enter credentials, submit, and validate the result.
- ✓ Assert both the success (logged-in) and failure (error message) paths.
- ✓ Recognise why brittle CSS/XPath locators cause maintenance pain later.

A locator is a description of how to find an element. Playwright does not grab the element once — it re-finds it on every action, which is what makes tests resilient.



The DOM

The browser's live tree of elements (buttons, inputs, links). A locator points into this tree by role, text, label or attribute.



Lazy & re-evaluated

A locator is a 'how to find', not the element itself. It is resolved fresh on each action — no stale references.



From Java/Selenium: a Selenium WebElement is fetched once and can go stale; a Playwright Locator is re-resolved each time — like a By that never expires.

This is also why you rarely need explicit waits in Playwright (Day 3).

If you saw Selenium in college, the concepts map directly. The names and the default priority differ.

SELENIUM (JAVA)	PLAYWRIGHT
By.id / By.name	→ <code>getByLabel</code> / <code>getByRole</code>
By.cssSelector	→ <code>page.locator('css=...')</code>
By.xpath	→ <code>page.locator('xpath=...')</code> (last resort)
<code>driver.findElement(By...)</code>	→ <code>page.getByRole(...)</code> / <code>page.locator(...)</code>
<code>element.sendKeys('text')</code>	→ <code>locator.fill('text')</code>
<code>element.click()</code>	→ <code>locator.click()</code>
<code>new Select(el).selectByVisibleText</code>	→ <code>locator.selectOption({ label })</code>
WebElement (can go stale)	→ <code>Locator</code> (re-resolved, no staleness)

Key shift: Selenium teaches id/CSS/XPath first; Playwright recommends user-facing locators first, CSS/XPath as a last resort.

06 Locator Priority

CONCEPT

Use the most user-facing, resilient locator available. Reach down the list only when the one above is not possible.

1 **getByRole** Accessible role + name — most resilient, mirrors how users/AT perceive the UI

2 **getByLabel** Form fields by their label — ideal for inputs

3 **getByText** Visible text — good for non-form elements

4 **getByTestId** Stable data-testid hook for dynamic UI

5 **CSS selector** Structural — breaks when markup changes

6 **XPath** Last resort — most brittle, hardest to read

Rule of thumb: if a human refers to it by what it says or does, use a user-facing locator.

Locators resolve against the DOM — the browser's tree of elements. A login form is a small branch of that tree.

```
<form aria-label="Login">
  <label>Email
    <input type="email"/>
  </label>
  <label>Password
    <input type="password"/>
  </label>
  <button>Sign in</button>
</form>
```

How Playwright sees it

role: **textbox**

name 'Email'

role: **textbox**

name 'Password'

role: **button**

name 'Sign in'

Semantic HTML gives every control a role and an accessible name — exactly what `getByRole` targets.


```
// 1. role + accessible name (preferred)
page.getByRole('button', { name: 'Sign in' });

// 2. form field by label
page.getByLabel('Email');

// 3. visible text    4. stable test id
page.getByText('Forgot password?');
page.getByTestId('login-error');

// 5/6. CSS / XPath — only when nothing above works
page.locator('css=.login-form button');
```

Same shape every time: `page.getByX(...)`. Also useful: `getByPlaceholder`, `getByAltText`, `getByTitle` — when role/label/text don't fit.

09 Input Handling

HANDS-ON

The handful of interactions you will use in almost every test:

```
// text input – clears the field, then sets the value (preferred)
await page.getByLabel('Email').fill('user@test.com');

// checkbox / radio
await page.getByLabel('Remember me').check();

// dropdown by visible label
await page.getByLabel('Country').selectOption({ label: 'India' });

// click a button
await page.getByRole('button', { name: 'Sign in' }).click();
```



From Selenium: `fill()` replaces `sendKeys()` and clears the field first; `selectOption()` replaces the `Select` helper class.

fill() – default

- Clears the field, then sets the value in one step
- Fast and stable
- Use for almost all inputs
- `locator.fill('value')`

pressSequentially()

- Types one character at a time
- Use only when the UI reacts per keystroke (autocomplete, masks)
- Slower; not needed for normal inputs
- `locator.pressSequentially('value')`

Default to fill(). Only reach for per-key typing when the application actually needs it.

11 Anatomy of a Login Form

CONCEPT

A typical login form has four things to automate. Map each to a user-facing locator first.



Email / username field

```
getByLabel('Email')
```



Password field

```
getByLabel('Password')
```



Submit button

```
getByRole('button', { name: 'Sign in' })
```



Error message

```
getByRole('alert') or getByTestId('login-error')
```

12 Automating the Login

HANDS-ON



IN THE IDE create tests/login.spec.ts and build this with me line by line.

```
import { test, expect } from '@playwright/test';

test('valid user can log in', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('user@test.com');
  await page.getByLabel('Password').fill('Secret123');
  await page.getByRole('button', { name: 'Sign in' }).click();
  // validate the logged-in state
  await expect(page.getByRole('heading', { name: /welcome/i })).toBeVisible();
});
```

Pattern: navigate → fill by label → submit by role → assert the logged-in state (not just “no error”).

13 Run It & Read the Result

HANDS-ON



IN THE IDE run headed first to watch the form fill, then headless; open the report together.

```
npx playwright test login.spec.ts --headed
```

Watch it fill & submit

```
npx playwright test login.spec.ts
```

Run headless (faster)

```
npx playwright show-report
```

Open the HTML report

What to look for: the test drives the real form, lands on the logged-in page, and the assertion goes green.

14 Assertion Strategies

CONCEPT

A login test is only trustworthy if it checks the right thing on both paths.



Success path

Assert a logged-in signal:

- welcome heading / avatar visible
- URL moved to /dashboard
- log-out control present



Failure path

Assert the error is shown:

- error message text visible
- still on /login
- no logged-in signal present

```
// failure path
await expect(page.getByRole('alert')).toContainText(/invalid credentials/i);
```

Playwright runs locators in strict mode: if a locator matches more than one element, the action fails instead of guessing. This catches ambiguous selectors early.

```
// ✗ throws if there are two 'Submit' buttons
await page.getByRole('button', { name: 'Submit' }).click();

// ✓ narrow the scope to the right form
await page.getByRole('form', { name: 'Login' })
    .getByRole('button', { name: 'Submit' }).click();

// ✓ or be more specific with an exact name
await page.getByRole('button', { name: 'Submit', exact: true });
```

Fix the locator — do not reach for `.first()` to silence it unless you truly mean 'any of these'.

GUIDED PRACTICE

Guided Exercises

Three short 'you try' tasks on today's material. Attempt each in the IDE, then we review together.

~30 minutes · individual · ask for help anytime

Exercise 1 — Fix the Brittle Locator

YOU TRY

This selector targets the Sign in button by CSS structure. It breaks the moment the markup changes. Replace it with a user-facing locator.

Your task:

- Rewrite the locator using getByRole with the button's name.
- Confirm it still clicks the same button.



IN THE IDE edit login.spec.ts — swap the CSS selector for a role-based one.

```
// brittle – depends on layout
await page.locator('css=.form > div:nth-child(3) > button').click();
```

18 Exercise 1 – Solution

SOLUTION

```
// resilient – by role + accessible name  
await page.getByRole('button', { name: 'Sign in' }).click();
```

Why: the role-based locator survives layout changes — the button keeps its role and label even if the surrounding markup is rearranged. The nth-child CSS breaks on any structural edit.



From Selenium: this is the difference between a fragile By.cssSelector chain and locating by what the user actually sees.

19 Exercise 2 – Fill & Submit

YOU TRY

Complete the login steps. The navigation and assertion are done — fill in the three interaction lines.

Your task:

- Fill Email and Password by label.
- Click Sign in by role.



IN THE IDE fill in the three TODO lines in login.spec.ts.

```
await page.goto('/login');  
// TODO: fill email, fill password, click Sign in  
await expect(page.getByRole('heading', { name: /welcome/i })).toBeVisible();
```

```
await page.goto('/login');  
await page.getByLabel('Email').fill('user@test.com');  
await page.getByLabel('Password').fill('Secret123');  
await page.getByRole('button', { name: 'Sign in' }).click();
```

Why: label-based locators for the inputs and a role-based locator for the button — every line targets what the user sees, none depend on CSS structure.

21 Exercise 3 – The Failure Path

YOU TRY

This test logs in with a wrong password but only checks that it is still on /login. That is weak — the page could be broken in other ways. Strengthen it.

Your task:

- Add an assertion that the error message is actually shown.
- Use a user-facing locator for the error (role alert or a test id).



IN THE IDE extend the invalid-login test with an error assertion.

```
// weak – only checks the URL  
await expect(page).toHaveURL(/\/login/); // TODO: assert the error too
```

22 Exercise 3 – Solution

SOLUTION

```
// still on /login (precondition)
await expect(page).toHaveURL(/\/login/);

// the real check – the error is shown to the user
await expect(page.getByRole('alert')).toContainText(/invalid credentials/i);

// optional, stronger – confirm NOT logged in
await expect(page.getByRole('heading', { name: /welcome/i })).not.toBeVisible();
```

⚠ **Lesson:** always assert what the user should see on failure — 'still on the page' alone does not prove the error was handled.

PROJECT WORK

Afternoon Lab

Build the login workflow yourself, in five staged milestones. Each has an acceptance gate before you move on.

~4 hours · work in the IDE · trainer circulates

24 Lab Overview

PROJECT WORK

Goal: a login spec that covers the success and failure paths, using user-facing locators and no secrets in the repo.

1 Build the login spec

Navigate + locate fields by label, button by role

2 Success path

Valid login asserts a logged-in signal

3 Failure path

Invalid login asserts the error message

4 User-facing locators

Replace any CSS/XPath where possible

5 Secrets out of repo

Credentials from env var / test-users file

Create the spec and locate the form's controls before automating anything.



IN THE IDE create tests/login.spec.ts; navigate to /login and locate the three controls.

```
test('valid user can log in', async ({ page }) => {  
  await page.goto('/login');  
  const email = page.getByLabel('Email');  
  const password = page.getByLabel('Password');  
  const signIn = page.getByRole('button', { name: 'Sign in' });  
  await expect(email).toBeVisible();  
  await expect(password).toBeVisible();  
  await expect(signIn).toBeVisible();  
});
```

✓ **Gate:** all three controls resolve and are visible (no strict-mode errors).

26 Milestone 2 — Success Path

LAB

Automate a valid login and assert a positive logged-in signal — not just the absence of an error.



IN THE IDE declare the locators, fill, submit, then assert a logged-in signal.

```
test('valid user can log in', async ({ page }) => {  
  await page.goto('/login');  
  const email = page.getByLabel('Email');  
  const password = page.getByLabel('Password');  
  const signIn = page.getByRole('button', { name: 'Sign in' });  
  await email.fill('user@test.com');  
  await password.fill('Secret123');  
  await signIn.click();  
  await expect(page.getByRole('heading', { name: /welcome/i })).toBeVisible();  
});
```

✓ **Gate:** valid login passes; the assertion targets a logged-in signal.

27 Milestone 3 – Failure Path

LAB

Add a second test for invalid credentials and assert the error message is shown.



IN THE IDE add a test with a wrong password; assert the alert/error text.

```
test('invalid login shows an error', async ({ page }) => {  
  // ... goto, fill email, fill wrong password, click Sign in  
  await expect(page.getByRole('alert')).toContainText(/invalid/i);  
});
```

✓ **Gate:** invalid login asserts the visible error; the test fails if the error is missing (prove it).

Audit your spec: every locator should be user-facing where the app allows it.

- Replace any `page.locator('css=...')` / XPath with `getByRole/getByLabel/getByText`.
- Where a hook is unavoidable, prefer a data-testid over a structural selector.
- Re-run both tests to confirm nothing regressed.



IN THE IDE `grep` your spec for 'css=' and 'xpath=' — replace what you can.

```
# quick audit
grep -nE "css=|xpath=" tests/login.spec.ts
```

✓ **Gate:** no structural CSS/XPath left except where genuinely unavoidable (and justified).

Never hard-code real credentials. Read them from the environment so nothing sensitive lands in Git.



IN THE IDE read credentials from env with a clear guard; reference them in the spec.

```
// read from environment, not hard-coded
const EMAIL = process.env.TEST_EMAIL;
const PWD = process.env.TEST_PASSWORD;

if (!EMAIL || !PWD) {
  throw new Error('Set TEST_EMAIL and TEST_PASSWORD');
}
await page.getByLabel('Email').fill(EMAIL);
```

✓ **Gate:** no credentials in the spec; a missing env var fails fast with a clear message.

How to provide the values when you run the tests — pick your shell:

macOS / Linux (bash, zsh)

```
TEST_EMAIL=user@test.com TEST_PASSWORD=Secret123 \  
npx playwright test login.spec.ts
```

Windows PowerShell

```
$env:TEST_EMAIL="user@test.com"  
$env:TEST_PASSWORD="Secret123"  
npx playwright test login.spec.ts
```

Tip: the trainer repo ships a git-ignored .env with test-user values, loaded via dotenv in the config.

31 Common Pitfalls

WATCH-OUTS



CSS/XPath first

Reaching for selectors before role/label — brittle and unreadable.



Asserting 'no error'

Always assert a positive logged-in signal, not just absence of an error.



Index-based picks

nth-child / [2] break the moment the layout shifts.



Ignoring strict mode

Using .first() to mute a multi-match instead of fixing the locator.



Hard-coded waits

Do not sleep before clicking submit — assertions auto-wait (Day 3).



Secrets in code

Use env vars / test users, never real credentials in the repo.

Debrief — Is Your Login Test Solid?

Before you call the login spec done, sanity-check it against these.

- ✓ **Both paths covered?** Success and failure each have their own test.
- ✓ **Positive signal asserted?** Success checks a logged-in element, not 'no error'.
- ✓ **Error actually asserted?** Failure checks the visible error, not just the URL.
- ✓ **Locators user-facing?** role / label / text over CSS / XPath.
- ✓ **No secrets committed?** Credentials come from env, not the spec.

Day 2 contribution areas for the Week 1 mini-capstone. The full rubric is scored out of 100 at the weekly pair demo — here is where each area comes from today:

Concept understanding	Explain locator priority & strict mode	15
Hands-on implementation	Working login spec in Git	30
Framework quality	User-facing locators, no brittle CSS/XPath	15
Assertions & coverage	Success + failure paths asserted	15
Headless / report readiness	Both paths pass headless + report	10
Git hygiene	Branch, commit, no secrets in repo	10
Collaboration & demo	Explain locator choices clearly	5
Total	Weekly mini-capstone score	100

- ✓ A locator is a 're-findable' description — not a one-time grab like a Selenium WebElement.
- ✓ Priority: `getByRole` → `getByLabel` → `getByText` → `getById` → CSS → XPath.
- ✓ Use `fill()` for inputs; `pressSequentially()` only when the UI reacts per keystroke.
- ✓ Automate login by label + role, then assert a positive logged-in signal.
- ✓ Always cover the failure path; respect strict mode by fixing ambiguous locators.
- ✓ Keep credentials out of the repo — read them from the environment.

35 Quick Reference

RECAP

Pin this — the locators and the login pattern you will reuse all week.

Locators (in priority)

```
getByRole('button',  
  { name: 'Sign in' })  
getByLabel('Email')  
getByText('Forgot?')  
getByTestId('login-error')  
// last resort  
locator('css=...')
```

Login pattern

```
const email = page  
  .getByLabel('Email');  
const password = page  
  .getByLabel('Password');  
const signIn = page.getByRole(  
  'button', { name: 'Sign in' });  
await email.fill(EMAIL);  
await password.fill(PWD);  
await signIn.click();
```

Day 2 complete

You can find elements like a user, fill and submit forms, and automate a login with success and failure assertions.



Next — Day 3: Synchronization — auto-waiting, dynamic elements, and eliminating flaky tests.